



UPY BIS
universities
**UNIVERSIDAD
POLITÉCNICA
DE YUCATÁN**

Classwork #15

Bubble Sort — Step by Step, Visualized

Dr. Jorge J. Pedrozo Romero

Data and Artificial Intelligence Engineering

2nd Quadrimester

May - August (2026)

Why Visualize an Algorithm?



UPY



Why Visualize an Algorithm?

❓ Why not just read the code?

- Code tells you **what** happens on one line at a time
- A visualization shows you **the shape of the process** — how order emerges from many small local decisions
- Sorting is one of the clearest algorithms to see: bars that are out of place move, one swap at a time, until the whole array "settles"

💡 Today: understand Bubble Sort by tracing it by hand first, then watch it happen with bars drawn using `std::draw`.

The Idea Behind Bubble Sort



UPY



The Idea Behind Bubble Sort

Core rule: compare two *neighboring* elements. If they are in the wrong order, swap them.

Repeat this sweep across the array. Every full sweep is called a **pass**.

✔ What happens	✘ What doesn't
Larger values slowly "bubble up" toward the end	Elements don't jump far in one step
Each pass guarantees one more element is in its final position	It's not the fastest algorithm — but it's the clearest to trace

≡ The name comes from how the largest unsorted value "bubbles" to its final position each pass — like a bubble rising in water.

The Rule, in Plain English (PPP style)

```
FOR each pass:  
  FOR each pair of neighbors (left, right) not yet finalized:  
    IF left > right:  
      swap(left, right)
```

⚠ Notice: **no new variables need to be invented** — this is just repeated comparison and conditional swap. That simplicity is exactly why it's the best first sorting algorithm to trace by hand.

Step-by-Step Trace

Full Array: [5, 3, 8, 4, 2]

Starting Array

Index:	0	1	2	3	4
Value:	5	3	8	4	2

► We will compare each neighboring pair, left to right, one pass at a time.

Pass 1 — Step 1: compare index 0 and 1

```
[ 5  3  8  4  2 ]  
  ^  ^
```

5 > 3 → SWAP

Result:

```
[ 3  5  8  4  2 ]
```

Pass 1 — Step 2: compare index 1 and 2

```
[ 3   5   8   4   2 ]  
    ^   ^  
5 > 8? No → no swap
```

Result (unchanged):

```
[ 3   5   8   4   2 ]
```

Pass 1 — Step 3: compare index 2 and 3

```
[ 3   5   8   4   2 ]  
      ^   ^
```

8 > 4 → SWAP

Result:

```
[ 3   5   4   8   2 ]
```

Pass 1 — Step 4: compare index 3 and 4

```
[ 3   5   4   8   2 ]  
           ^   ^
```

8 > 2 → SWAP

Result:

```
[ 3   5   4   2   8 ]
```

End of Pass 1

```
[ 3   5   4   2   8 ]  
          ---  
        finalized ✓
```

- ✓ The largest value (8) is now in its correct final position. The next pass never needs to check it again.

Pass 2 — Step 1: compare index 0 and 1

```
[ 3  5  4  2 | 8 ]  
  ^  ^  
3 > 5? No → no swap
```

Result (unchanged):

```
[ 3  5  4  2 | 8 ]
```

Pass 2 — Step 2: compare index 1 and 2

```
[ 3   5   4   2 | 8 ]  
      ^   ^
```

5 > 4 → SWAP

Result:

```
[ 3   4   5   2 | 8 ]
```

Pass 2 — Step 3: compare index 2 and 3

```
[ 3   4   5   2 | 8 ]  
      ^   ^
```

5 > 2 → SWAP

Result:

```
[ 3   4   2   5 | 8 ]
```


End of Pass 2

```
[ 3   4   2 | 5   8 ]  
          -----  
        finalized ✓
```

Two elements are now finalized (5 and 8). Pass 3 only needs to sweep the first 3 positions.

Pass 3 — Step 1: compare index 0 and 1

```
[ 3  4  2 | 5  8 ]  
  ^  ^  
3 > 4? No → no swap
```

Result (unchanged):

```
[ 3  4  2 | 5  8 ]
```

Pass 3 — Step 2: compare index 1 and 2

```
[ 3   4   2 | 5   8 ]  
      ^   ^
```

4 > 2 → SWAP

Result:

```
[ 3   2   4 | 5   8 ]
```

End of Pass 3

```
[ 3  2 | 4  5  8 ]  
-----  
  finalized ✓
```

Pass 4 — Step 1: compare index 0 and 1

```
[ 3  2 | 4  5  8 ]  
  ^  ^
```

3 > 2 → SWAP

Result:

```
[ 2  3 | 4  5  8 ]
```

Array Fully Sorted ✓

```
[ 2  3  4  5  8 ]
```

Total work for $n = 5$: 4 passes, 10 comparisons, 5 swaps.

💡 Notice the pattern: pass k only needs to check $n - k$ pairs, because the last $k - 1$ elements are already finalized. This is the first optimization worth coding.

Complexity



UPY



Complexity

Case	Comparisons	Why
Worst case	$O(n^2)$	Array is reverse-sorted — every pair needs a swap, every pass
Best case	$O(n)$ (with an early-stop flag)	Array is already sorted — one pass detects no swaps and stops
Space	$O(1)$	Sorting happens in place — no extra array is created

💡 Optional extension: add a `swapped` flag that breaks out of the outer loop early if a full pass makes **zero swaps** — the array is already sorted, no need to keep going.